manifest.csv

Split by 'split' column
train_df / val_df / test_df

LABEL_TO_IDX mapping [labels.py]

Train set: PlaceholderDatasetChunked

chunk_clip() librosa.load full clip -> split into 4.0s non-overlapping windows

_loop_fill() trailing remainder forward-looped + crossfaded to a full 4.0s window

logmel() per chunk [preprocess.py]

cache chunk .npy (../data/cache_chunked)

per-sample normalize zero-mean / unit-std

Val/Test set: PlaceholderDatasetNatural

librosa.load full clip sr=32000, mono=True (no crop / pad / loop)

logmel() [preprocess.py]

cache clip .npy (../data/cache_natural)

per-sample normalize zero-mean / unit-std

Training setup

CNNBaseline() [models/cnn.py]

criterion = CrossEntropyLoss()

optimizer = Adam lr=1e-3, weight_decay=1e-4

scheduler = ReduceLROnPlateau mode=max, factor=0.5, patience=3, min_lr=1e-5

train_loader batch_size=32, shuffle=True

val_loader / test_loader batch_size=1, shuffle=False (variable-width tensors can't batch-collate)

Epoch loop (N_EPOCHS=20)

model.train() for each train batch: zero_grad -> forward -> CE loss -> backward -> optimizer.step()

accumulate train_loss, train_acc

model.eval(), no_grad forward over val_loader

compute val_loss, val_acc, val_macro_f1 (sklearn f1_score)

scheduler.step(val_macro_f1) halves LR if stalled

append history: train/val loss+acc, val_macro_f1, lr

val_macro_f1 > best so far?

yes

no

torch.save checkpoint {state_dict, labels, val_macro_f1, epoch}

Best checkpoint: cnn_baseline_best.pt (highest val_macro_f1)